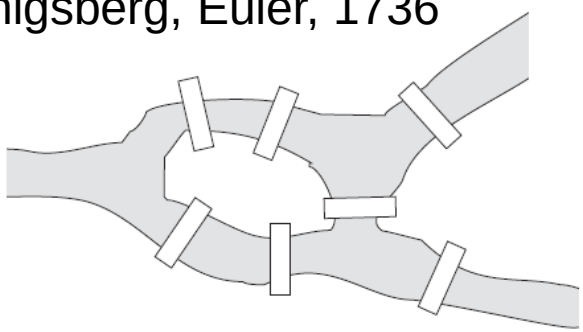
Föreläsning 11

Grafer, Dijkstra och Prim

Föreläsning 11

- Grafer
- Representation av grafer
- Dijkstras algoritm
- Implementation av Dijkstras algoritm
- Minimium spanning tree

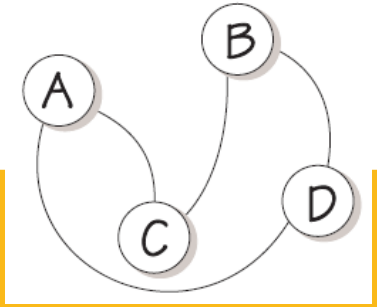
Broarna i Königsberg, Euler, 1736



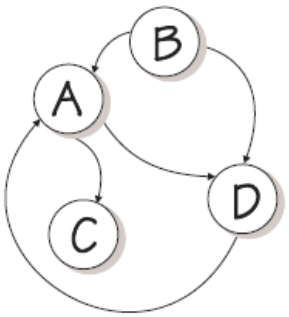
Grafer

Grafteori är en gren inom matematiken som är mycket användbar inom datalogi. Vi ska istället för att gå igenom boken ge en kort översikt. Se till att själv läsa kapitel 10 och göra uppgifter.

Graf (graph) - En mängd V av noder (hörn, vertices) och en mängd E av bågar (kanter, edges) sådan att ändpunkterna på varje båge i E finns i V. En graf skrivs G = (V, E). Ett exempel: G = ({A, B, C, D}, {(A, C), (A, D), (B, C), (B, D)})



Terminologi



Riktad graf (*digraf*) en graf där varje båge är riktad

Viktad graf (*weighted graph*) en graf där varje båge tilldelats ett tal som kallas för vikten, kostnaden eller avståndet.

Angränsande (adjacent) Nod A är angränsande till nod B om det finns en båge från B till A.

Väg *(path)*, en sekvens av bågar som knyter samman en sekvens av noder där varje nod är angränsande till noden före.

Enkel väg (*simple path*) en väg där varje båge och nod är unik utom den första och sista noden som får vara samma.

Cykel (cycle) En enkel väg där första och sista noden är samma.

Sammanhängande graf *(connected graph)* betyder i en icke riktad graf att det finns en väg mellan alla par av noder.

Brygga *(bridge*) är en båge, sådan att om den tas bort så är inte längre grafen sammanhängande.

Komplett graf (*complete graph*) en graf där varje par av noder har en gemensam båge.

Isomorfi (*isomorphic*) Två grafer med samma matematiska struktur (samma grannmatris).

Ordning (*order*) antal noder i grafen

Gradtal (*degree*) Givet en nod v, säger vi att *deg*(v) är antalet bågar som är anslutna till denna nod v.

Avstånd (*distance*) Givet två noder v och w den kortaste vägen *d(v,w)* mellan dessa noder.

Diameter (*diameter*) Det största avståndet *d(v,w)* som finns i en sammanhängande graf.

Eulerväg (*Eulerian path)* En väg som går genom samtliga grafens bågar precis en gång.

Hamilton cykel (*Hamilton cycle*) En cykel som besöker samtliga grafens noder precis en gång.

Träd En sammanhängande graf utan cykler

Skog (*forest*) en graf utan cykler (slutna vägar)

Exempel på tillämpningar

Flödesscheman

Sociala nätverk

Vägnät

Datornätverk

Transporter

Kartografi - att måla kartor. Noderna är länder och bågarna är gränser.

Binära sökträd (ett träd är en sammanhängande graf utan cykler)

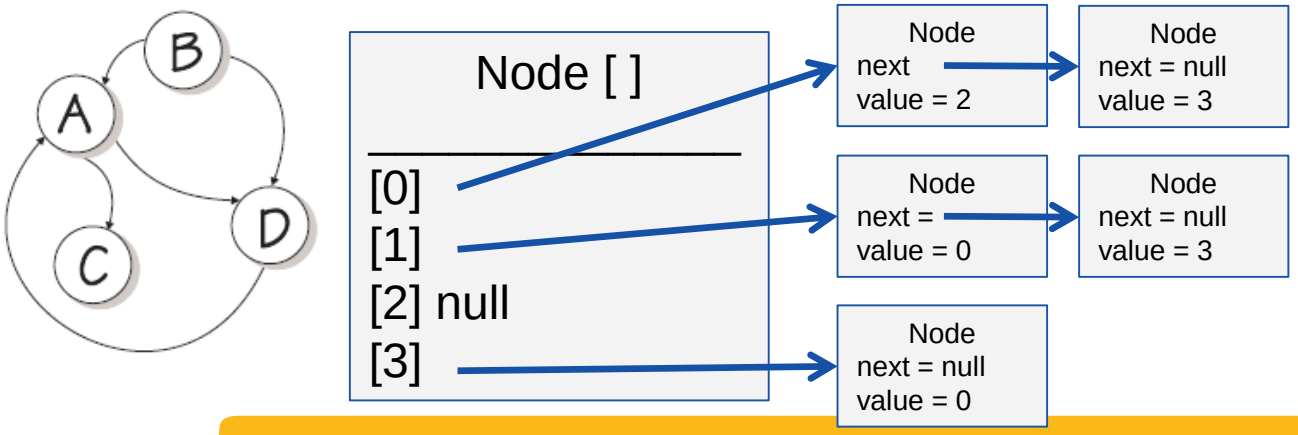
Implementering - representation

Vilka operationer en graf ADT behöver ha är starkt beroende på vad man ska ha grafen till. Boken går igenom ett ganska generellt exempel. Oftast designar man dock grafen efter vad man ska använda den till (Java tillhandahåller ingen generell). Oberoende av operationer behöver vi representera grafen. Det finns då två huvudalternativ:

- **Grannlista (förbindelselista, adjacency list)**
  En lista över listor på varje nods grannar
- **Grannmatris (förbindelsematris, adjacency matrix)**
  Varje rad/kolumn representerar en nod och skärningen representerar bågen, lite som en avståndstabell i en kartbok

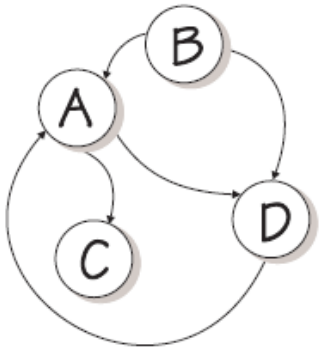
Grannlista

Grundiden att ha en lista av listor över varje nods grannar kan implementeras på många sätt. Om det är en viktad graf måste den också hålla reda på vikten för varje båge. I jämförelse med en grannmatris spar den oftast mycket utrymme då de flesta noder normalt inte har en granne. Den är också snabbare om man vill gå igenom en nods grannar (men något långsammare om man vill veta om två specifika noder är grannar). Vi visar här ett enkelt exempel med en riktad graf. A representeras av 0, B av 1 osv.



Vi kan också använda en färdig lista istället för att skapa en egen:

Grannlista mha färdiga listor



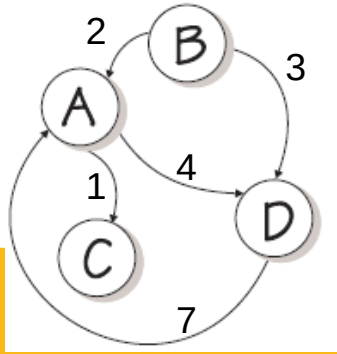

```
List<Integer>[] nodes = new ArrayList[4];
nodes[0] = new ArrayList<>();
nodes[0].add(2);
nodes[0].add(3);
nodes[1] = new ArrayList<>();
nodes[1].add(0);
nodes[1].add(3);
nodes[2] = new ArrayList<>();
nodes[3] = new ArrayList<>();
nodes[3].add(0);
```

Valde här en ArrayList men kunde också valt en länkad lista.

Grannlista med vikter

Eftersom varje plats i min array innehåller en lista med grannar till denna nod behöver jag inte spara startnoden för bågen utan bara slutnod och vikt. För att kunna spara två data i listan behöver jag skapa en klass som för varje båge innehåller slut-nod och vikt.

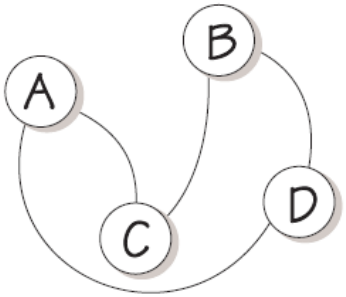
```
private static class Edge {

    private final int endNode, weight;

    private Edge(int v, int w) {
        endNode = v;
        weight = w;
    }
}
```



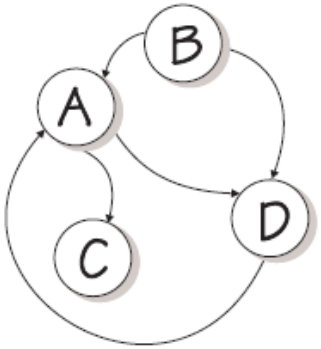

```
List<Edge>[] nodes = new ArrayList[4];
nodes[0] = new ArrayList<>();
nodes[0].add(new Edge(2,1));
nodes[0].add(new Edge(3,4));
nodes[1] = new ArrayList<>();
nodes[1].add(new Edge(0,2));
nodes[1].add(new Edge(3,3));
nodes[2] = new ArrayList<>();
nodes[3] = new ArrayList<>();
nodes[3].add(new Edge(0,7));
```

Grannmatris

En grannmatris har lika många rader och kolumner som antalet noder i grafen. Skärningen representerar bågen (0 ingen båge och 1 båge). I stora grafer har oftast de flesta noder ingen båge så matrisen som blir antal noder i kvadrat stor kommer att slösa mycket utrymme. Detta kan avhjälpas med olika tekniker för att spara glesa matriser eller genom att använda en hash-tabell där nyckeln är ett nod-par. I en viktad graf spar man helt enkelt vikten istället för ett i matrisen och Double.POSITIVE_INFINITY istället för 0.



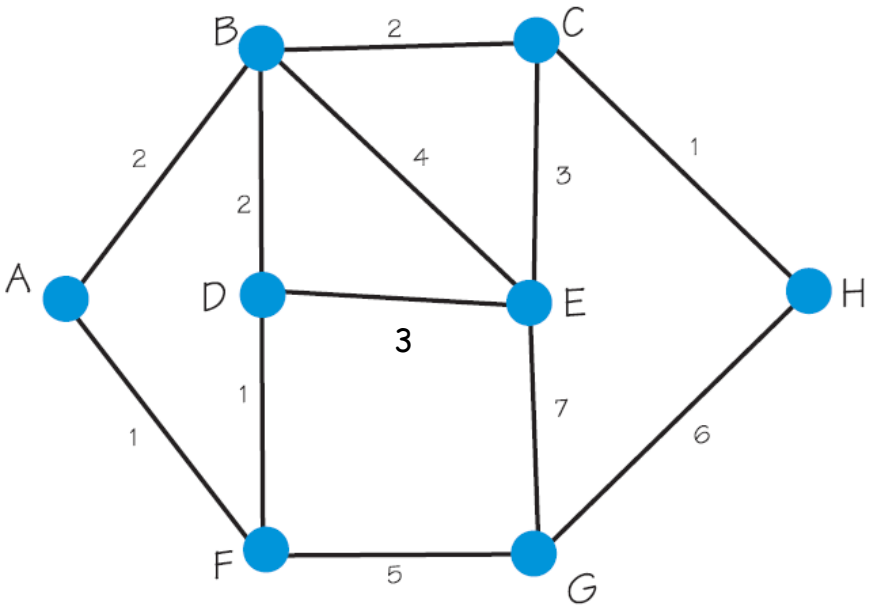
	A	B	C	D
A	0	0	1	1
B	0	0	1	1
C	1	1	0	0
D	1	1	0	0



	A	B	C	D
A	0	0	1	1
B	1	0	0	1
C	0	0	0	0
D	1	0	0	0

Dijkstras algoritm

Problem: Hitta den kortaste vägen från en given nod till alla andra noder i en viktad sammanhängande graf (med positiva vikter).



Dijkstras algoritm

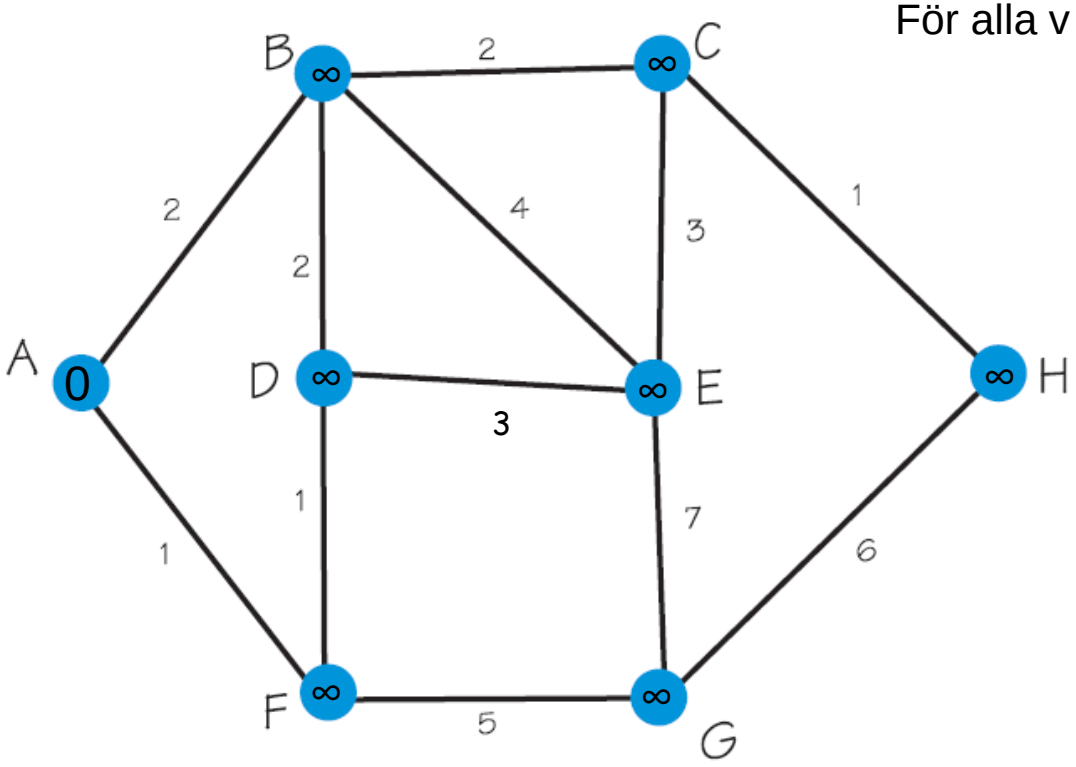
Input:
V – mängden av noder, s – startnoden
w(i,j) – vikt för båge från nod i till nod j

Output:
p(v) – noden före v i vägen från s till v
d(v) – kortaste vägen från s till v

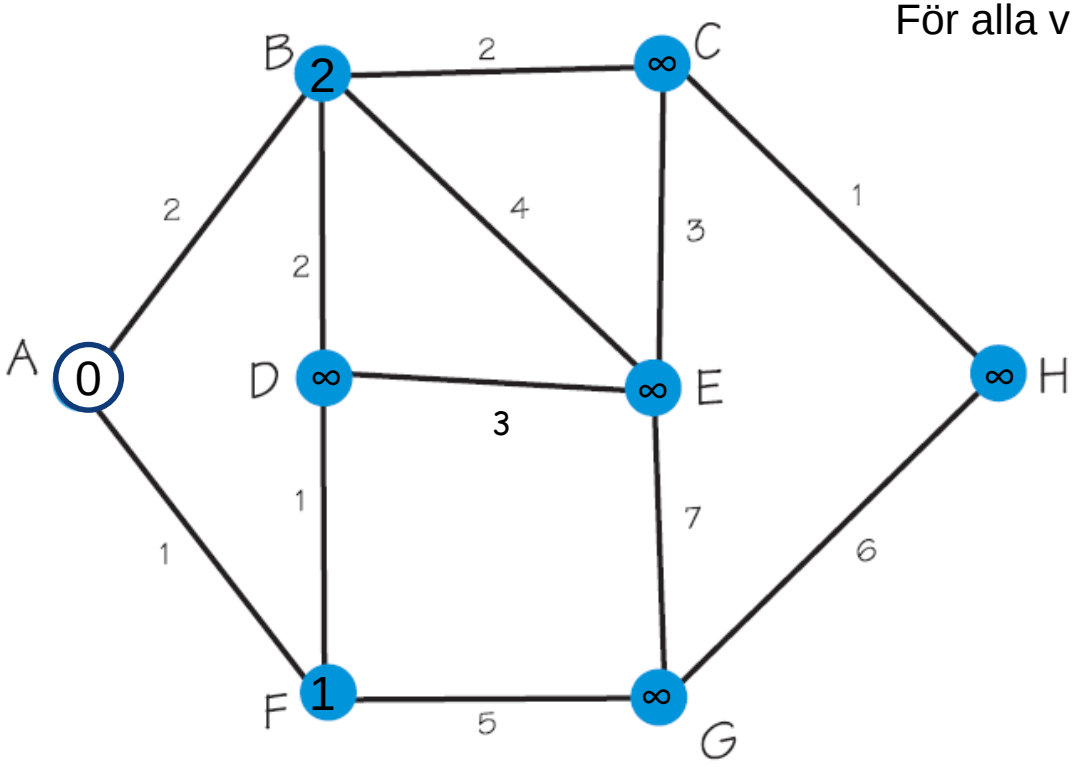
Sätt T = V
För alla $v \in T$ sätt $d(v) = \infty$
Sätt d(s) = 0
Medans $T \neq \emptyset$
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u

Egentligen räcker det att titta på de angränsande noderna i T men beroende på implementering kostar det inget extra att titta på alla angränsande

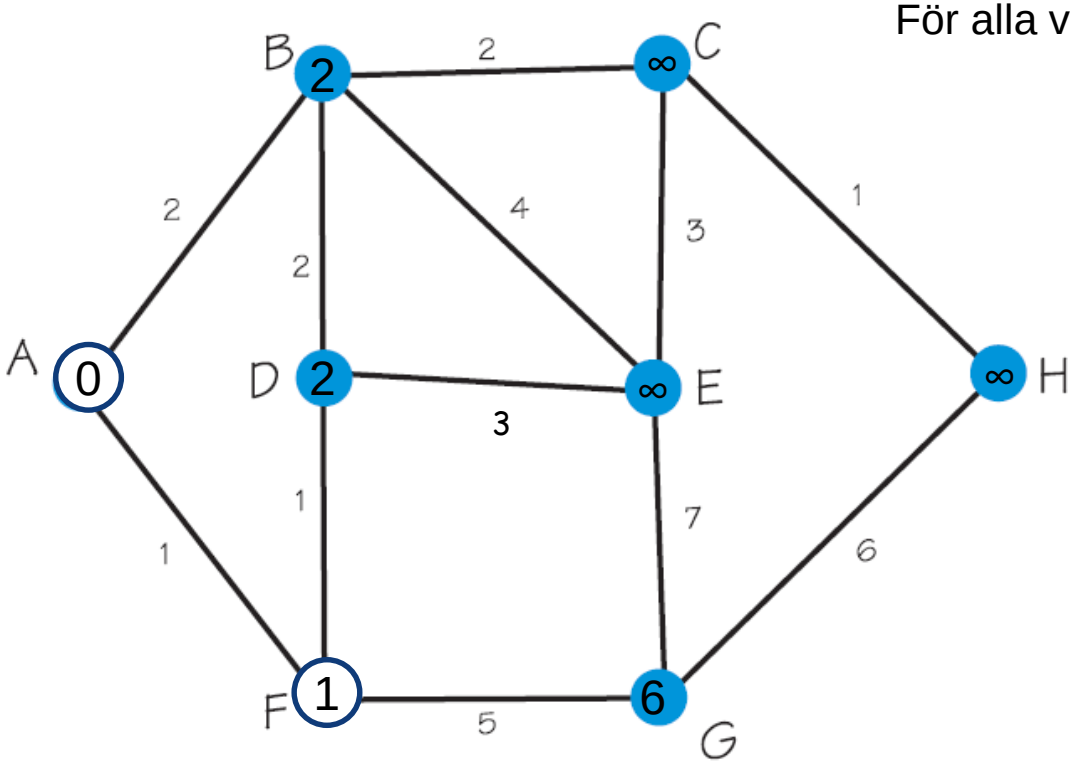
Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



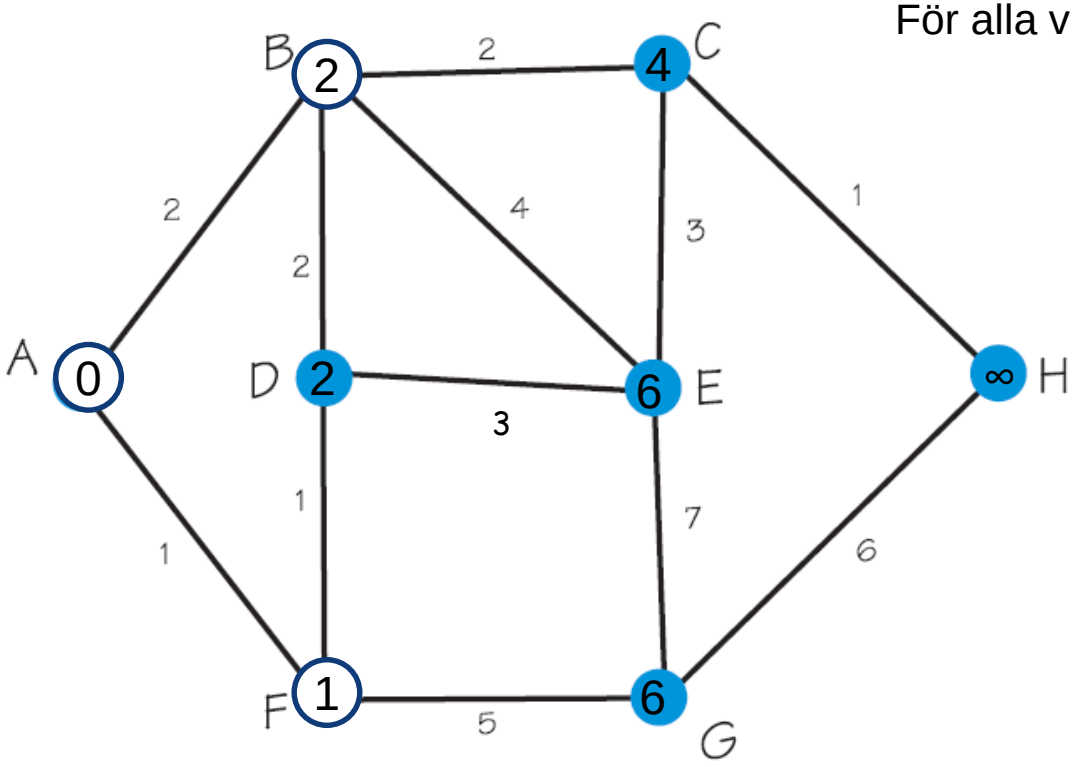
Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



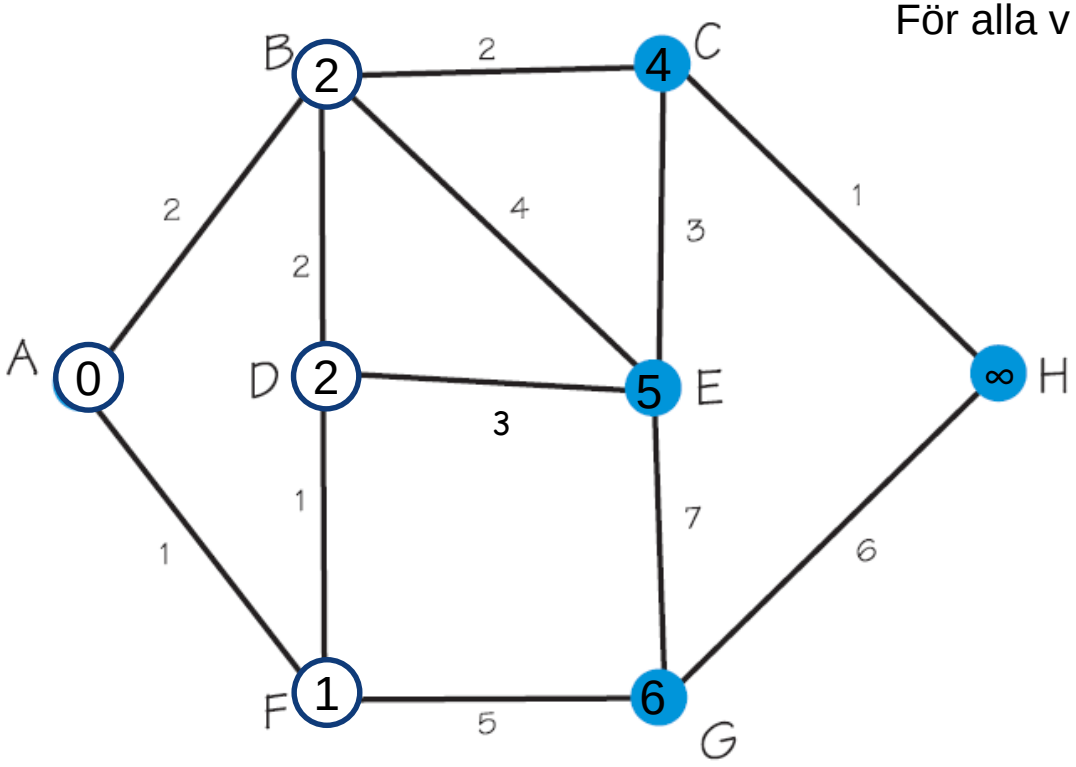
Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



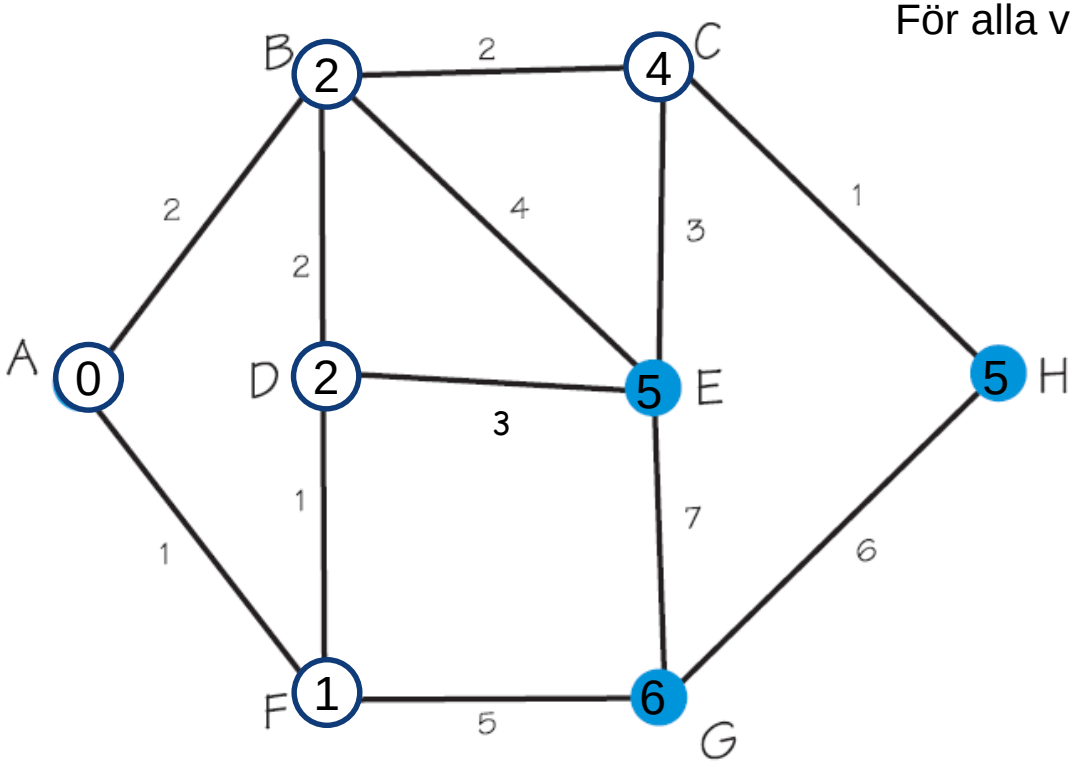
Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



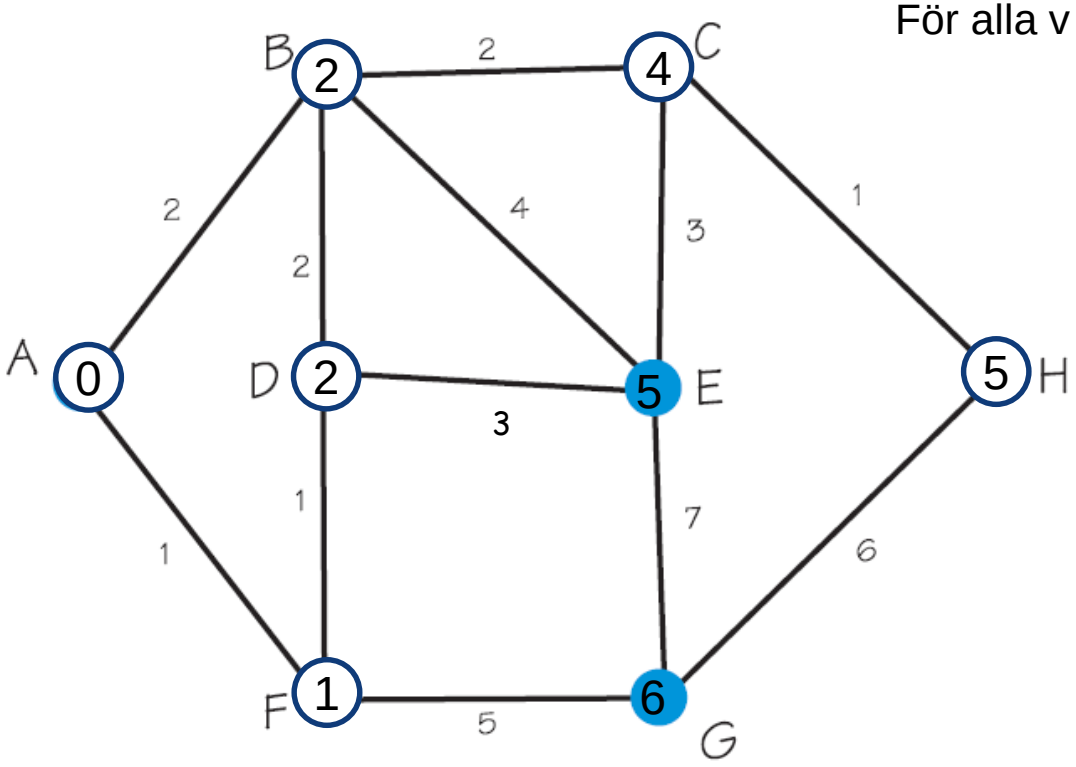
Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



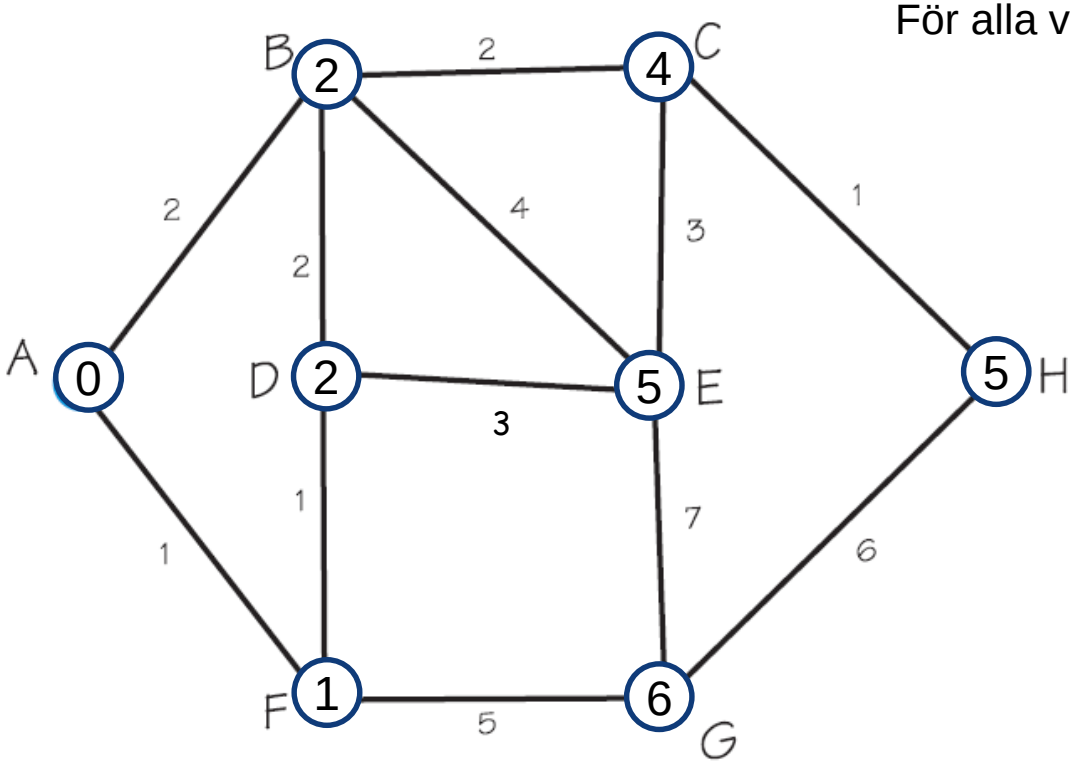
Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u



Loopen kommer att exekvera |V| gånger och gå igenom alla element i T varje gång för att hitta det minsta. Detta gör att algoritmen blir $O(|V|^2)$.

Observera att när en nod väl blivit vald kommer den inte uppdateras. Detta gör att om vi är intresserad av vägen till en specifik nod kan vi sluta när den blir vald

Implementering

För att snabbt komma igång med själva algoritmen skriver vi ett enkelt hårdkodat program med grannmatris som löser vårt exempel:

```
int X = Integer.MAX_VALUE;
int[][] w = {        {X, 2, X, X, X, 1, X, X},
                     {2, X, 2, 2, 4, X, X, X},
                     {X, 2, X, X, 3, X, X, 1},
                     {X, 2, X, X, 3, 1, X, X},
                     {X, 4, 3, 3, X, X, 7, X},
                     {1, X, X, 1, X, X, 5, X},
                     {X, X, X, X, 7, 5, X, 6},
                     {X, X, 1, X, X, X, 6, X}};
```

Här har vi då valt heltal som interna nodnamn: A – 0, B – 1, C – 2 osv
Fundera på hur du ska representera de övriga data vår algoritm behöver.

Input:
V – mängden av noder, s – startnoden
w(i,j) – vikt för båge från nod i till nod j
Output:
p(v) – noden före v i vägen från s till v
d(v) – kortaste vägen från s till v

```
Sätt T = V
För alla v∈T sätt d(v) = ∞
Sätt d(s) = 0
Medans T ≠ ∅
        Sätt u till en av noderna i T med minst d(u)
        T = T – {u}
        För alla v angränsande till u
                Om d(v)>d(u)+w(u,v)
                        sätt d(v) = d(u)+w(u,v)
                        sätt p(v) = u
```

Val av representation

För att vi ska kunna diskutera vidare föreslår jag att vi alla använder följande val:

```
int d[] = {0, X, X, X, X, X, X, X};
int p[] = new int [8];
boolean[] taken = new boolean[8];
//blir false
```

Output:
p(v) – noden före v i vägen från s till v
d(v) – kortaste vägen från s till v

Sätt T = V
För alla $v \in T$ sätt $d(v) = \infty$
Sätt d(s) = 0

Utgå nu från algoritmen och försök koda så nära denna som möjligt.

Medans $T \neq \emptyset$
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u

Snyggare och effektivare att använda ett BitSet för att representera T. Prova gärna.

Mitt förslag på kod

Observera att vi stannar när T innehåller en nod vilket är ok

```
for(int i= 0;i<d.length-1;i++){
        int u = next(d,taken);
        taken[u]=true;
        for(int v=0;v<w[u].length;v++)
                if(w[u][v]<X && d[v]>d[u]+w[u][v]){//utgår ifrån att w[u][u]=X
                        d[v]=d[u]+w[u][v];
                        p[v] = u;
        }
}
```

Medans T ≠ ∅
    Sätt u till en av noderna i T med minst d(u)
    T = T – {u}
    För alla v angränsande till u
        Om d(v)>d(u)+w(u,v)
            sätt d(v) = d(u)+w(u,v)
            sätt p(v) = u

next ska då leverera den nod (det index) som har minst värde på d och inte är tagen.

Observera att vi måste gå igenom hela raden när vi ska gå igenom alla angränsande. Här kan man spara tid med en grannlista där man bara går igenom faktiska grannar.

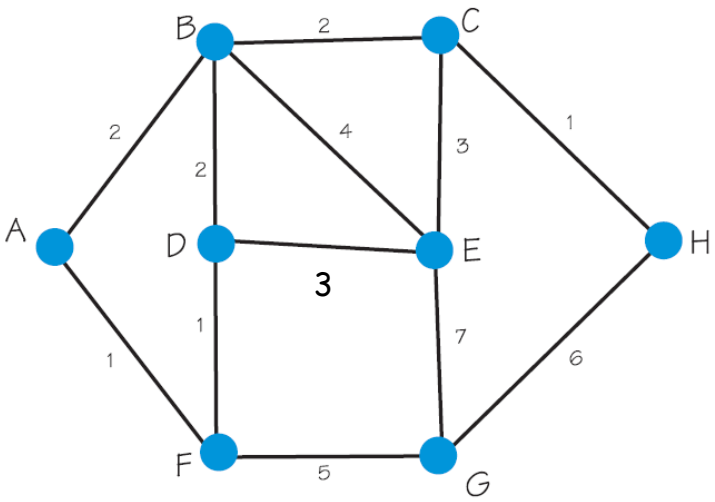
next

```
static int next(int[] d, boolean[] taken){
        int dMin=Integer.MAX_VALUE;
        int minIndex=0;
        for(int i=0;i<d.length;i++)
                if(d[i]<dMin&&!taken[i]){
                        dMin = d[i];
                        minIndex = i;
        }
        return minIndex;
}
```

Resultat

```
for(int i=1; i<d.length;i++)
  System.out.println("Nod "+(char)(i+'A')+" har avstånd "+d[i]+" till nod A via: "+(char)(p[i]+'A'));
```

Nod B har avstånd 2 till nod A via: A
Nod C har avstånd 4 till nod A via: B
Nod D har avstånd 2 till nod A via: F
Nod E har avstånd 5 till nod A via: D
Nod F har avstånd 1 till nod A via: A
Nod G har avstånd 6 till nod A via: F
Nod H har avstånd 5 till nod A via: C



Minsta uppspännande träd (Minimum Spanning Tree)

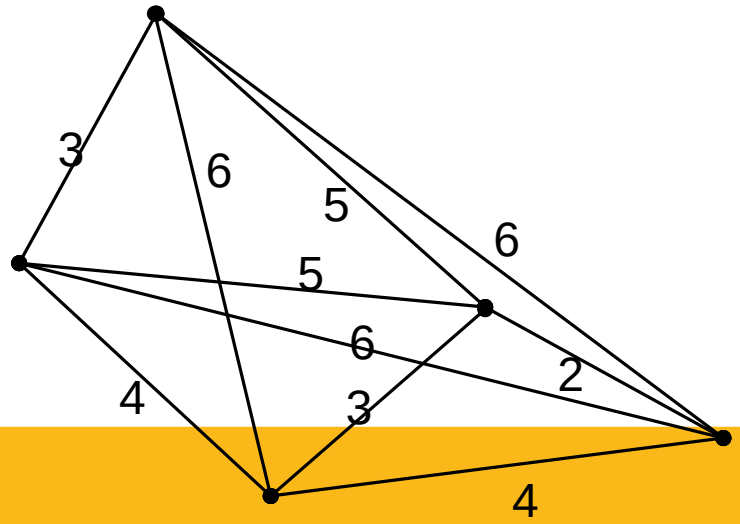
Ett uppspännande träd är en delmängd av bågarna till en sammanhängande graf sådan att det saknas cykler (träd) och inkluderar alla noder (uppspännande).

Problemet vi nu ska studera innebär att man i en viktad graf hittar det uppspännande träd som har minst total vikt på bågarna.

Som konkret exempel kan man tänka att man givet ett antal städer ska hitta det vägnät som ansluter alla städer för minsta möjliga kostnad.

Giriga algoritmer

En grupp algoritmer kallas giriga. En girig algoritm väljer den lokalt bästa lösningen i varje steg (den tar girigt vad den kan just nu). Sådana algoritmer hittar inte den bästa (korrekta) lösningen till alla problem men gör det till bland annat problemet ovan. Fundera på hur du skulle lösa problemet att bygga minsta möjliga vägnät mellan ett antal städer med hjälp av en girig algoritm.



Mer om giriga algoritmer i nästa föreläsning…

En girig algoritm

Välj godtycklig stad A

Förbind stad A med närmaste stad B

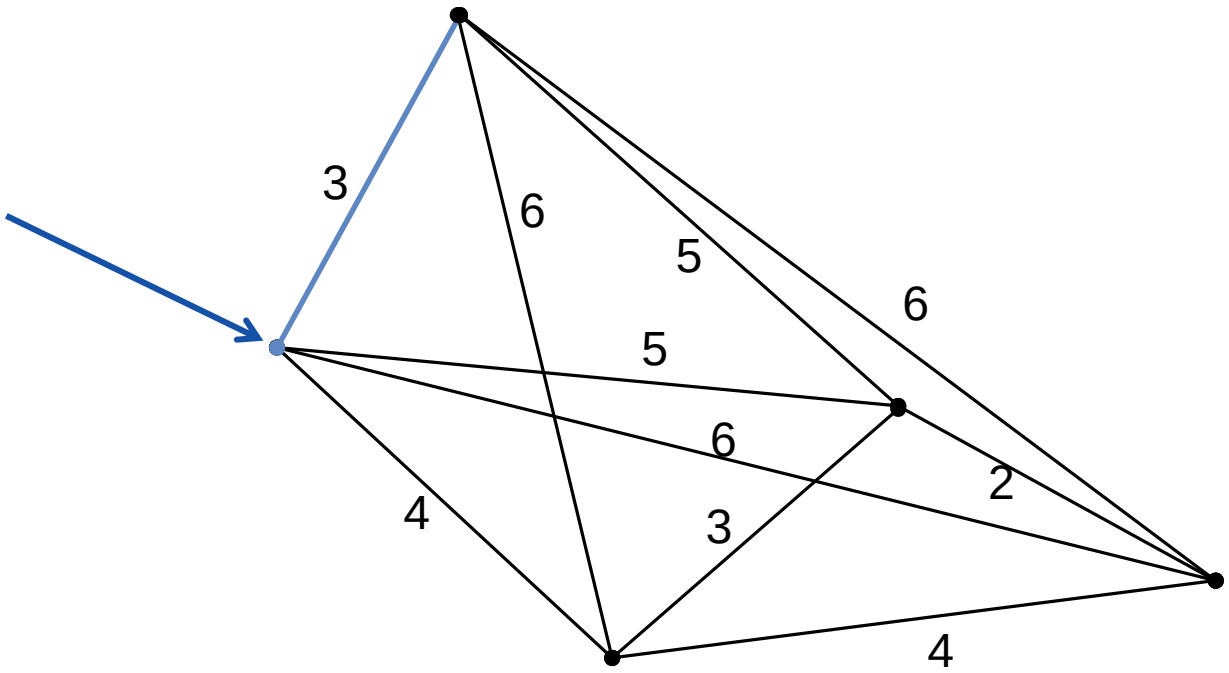
Sök upp den stad C som är närmast någon av städerna A och B

Förbind C med den stad A eller B som är närmast

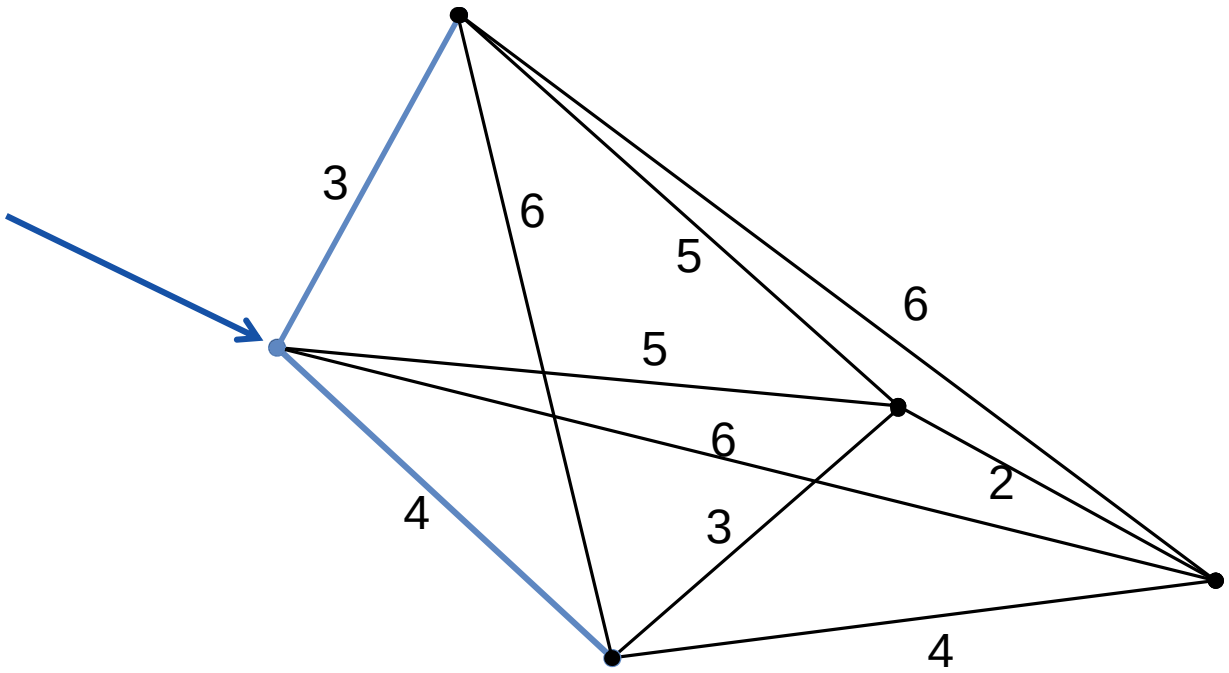
Sök upp den stad D som är närmast någon av A, B eller C

osv

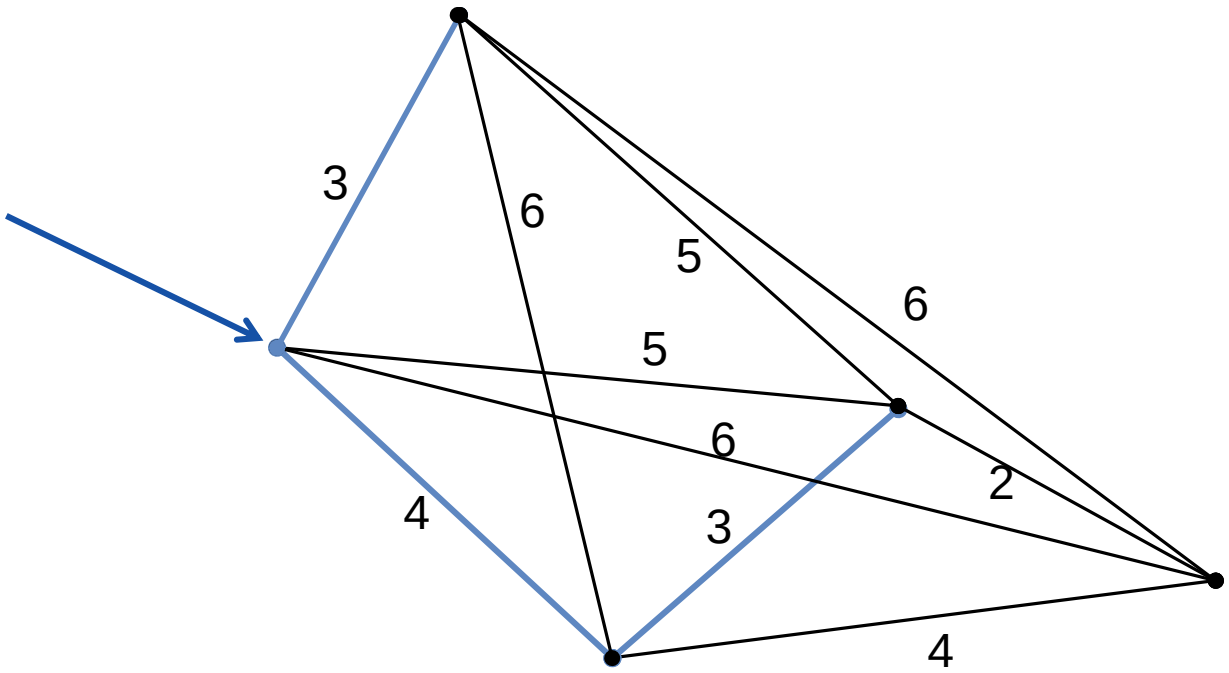
Ett exempel



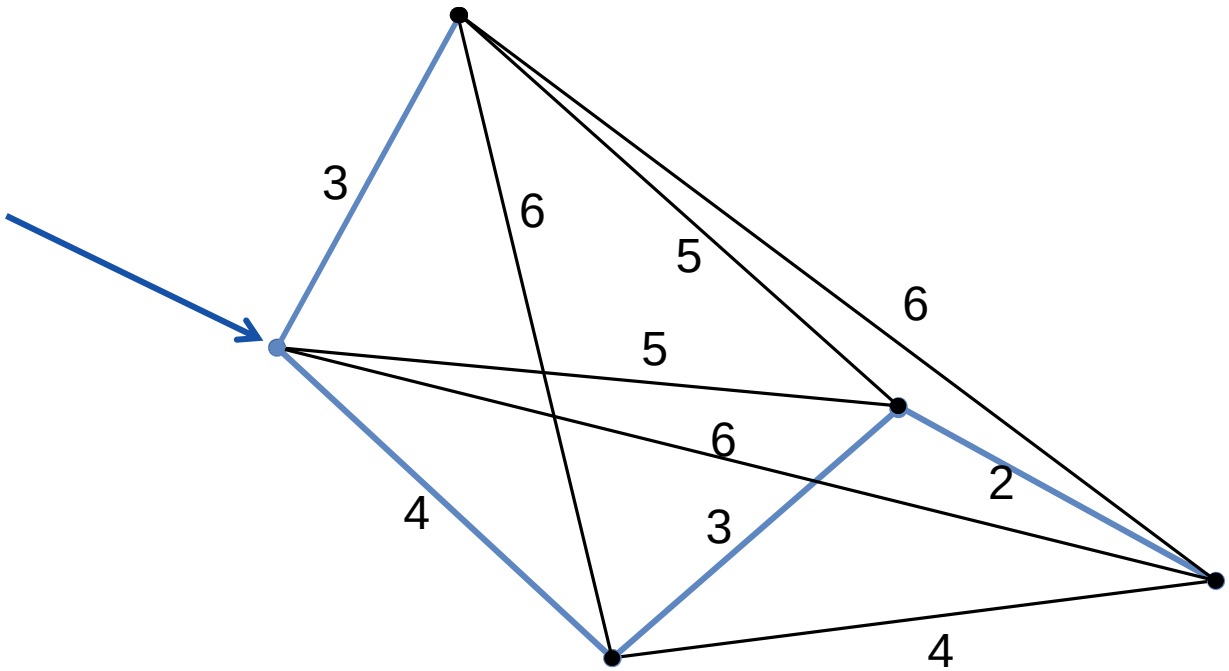
Ett exempel



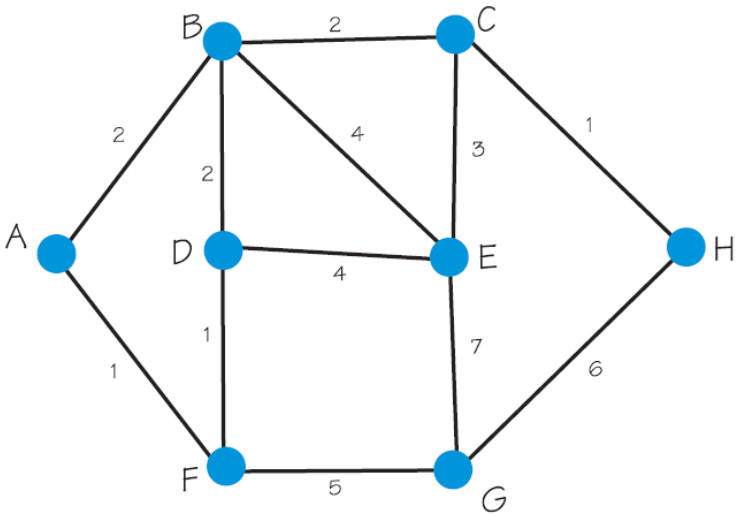
Ett exempel



Ett exempel



Prims algoritm



Algoritmen vi just beskrivit brukar kallas Prims algoritm (ibland Jarniks algoritm):

1. Välj en godtycklig nod och låt denna utgöra ett träd
2. Så länge det finns noder kvar att ansluta:
   Anslut en av de noder ännu inte ansluten som har kortast båge till trädet

Observera att denna algoritm är mycket lik dijkstra

Implementering prim

Observera att denna algoritm är mycket lik Dijkstras och kan implementeras på samma sätt. Du behöver endast göra några små ändringar till Dijkstras. Istället för att spara kortaste vägen till startnoden sparar du nu kortaste anslutningsvägen just nu.

```
for(int i= 0;i<d.length-1;i++){
        int u = next(d,taken);
        taken[u]=true;
        for(int v=0;v<w[u].length;v++)
                if(w[u][v]<X && d[v]>d[u]+w[u][v]){
                        d[v]=d[u]+w[u][v];
                        p[v] = u;
        }
}
```

Obs Dijkstra ej prim – behöver modifieras!

Komplexitet

Vår tillämpning av Dijkstra blir $O(V^2)$, där V är antalet noder. Om man använder en prioritetskö (Fibonacci heap) kan man komma ner i $O(E+V\log V)$, där E är antalet bågar. Implementerar man prim på samma sätt som Dijkstra, dvs om man sparar anslutningskostnaden för varje nod vid varje varv blir denna också $O(V^2)$ och också denna kan effektiviseras med en prioritetskö.

```
for(int i= 0;i<d.length-1;i++){
        int u = next(d,taken);
        taken[u]=true;
        for(int v=0;v<w[u].length;v++)
                if(w[u][v]<X && d[v]>d[u]+w[u][v]){
                        d[v]=d[u]+w[u][v];
                        p[v] = u;
        }
}
```